

Logging and Audit

Table of Contents

- [Overview](#)
- [Technical Summary](#)
- [Logging Implementation](#)
- [Audit Evidence](#)
- [Configuration Evidence](#)
- [Representative Code Evidence](#)
- [Risks and Observations](#)
- [Recommendations](#)

Overview

The repository contains a Java Spring Boot backend built with Gradle. Evidence in the project shows minimal, configuration-driven logging level entries in application.properties. There is no explicit, repository-level audit subsystem or audit-specific code identified in the provided materials. The documentation below is strictly constrained to artifacts and contents present in the repository and does not assume any unobserved runtime or operational integrations.

Technical Summary

Aspect	Value
Build system	Gradle (file: build.gradle)
Spring Boot plugin version	2.6.3 (build.gradle plugin declaration)
Java compatibility	11 (build.gradle: sourceCompatibility = '11')
Logging configuration source	application.properties (logging.level.* entries present)
Explicit logging configuration files (e.g., logback.xml)	Not found in repository evidence
Explicit logger usage in provided classes	No logger declarations or log statements present in the provided configuration classes
Audit-specific code or audit trail implementation	Not found in repository evidence
CI automation	GitHub Actions workflow at .github/workflows/gradle.yml
Maven build file	No pom.xml found at project root
Docker artifacts	No Dockerfile or compose files found

Logging Implementation

The repository contains only configuration-driven logging level entries found in `src/main/resources/application.properties`. The `application.properties` file includes two explicit logging level settings scoped to package/class patterns:

- `logging.level.io.spring.infrastructure.mybatis.readservice.ArticleReadService=DEBUG`
- `logging.level.io.spring.infrastructure.mybatis.mapper=DEBUG`

No explicit logback configuration file (for example `logback.xml` or `logback-spring.xml`) is present in the repository evidence. No alternative logging configuration files (e.g., `log4j2.xml`, `log4j2-spring.xml`) are present in the evidence set.

No explicit logger instantiation (for example `org.slf4j.LoggerFactory.getLogger(...)` or similar) or log statements were found in the provided configuration class sources (`WebSecurityConfig.java`, `JacksonCustomizations.java`, `MyBatisConfig.java`). The repository evidence does include Spring Boot and its starter dependencies in `build.gradle`; however, the presence of transitive logging libraries is not enumerated beyond what is present in `build.gradle` and `application.properties`. Statements in this section are limited to the explicit occurrences in repository files.

Audit Evidence

The repository does not contain explicit audit-related code or a dedicated audit subsystem in the provided evidence. Specifically:

- There are no classes, configuration files, or Mapper/Repository artifacts explicitly labeled or implemented as "audit" in the provided excerpts.
- The "EVIDENCE - AUDIT CODE:" marker in the supplied evidence is empty, and no alternative files referencing audit events, audit tables, audit interceptors, or audit log persistence were provided.

Conclusion (evidence-backed): Dedicated audit mechanisms are not clearly evidenced in the repository snapshot provided.

Configuration Evidence

The following configuration artifacts and settings are present and verified from repository files:

- `build.gradle`
 - Spring Boot plugin declaration: `org.springframework.boot` version 2.6.3
 - Java `sourceCompatibility/targetCompatibility` set to 11
 - Dependencies include: `org.springframework.boot:spring-boot-starter-web`, `spring-boot-starter-validation`, `spring-boot-starter-hateoas`, `spring-boot-starter-security`, `mybatis-spring-boot-starter`, `graphql-dgs`, `flyway-core`, `jjwt` (`api` / `runtime`), `joda-time`, `sqlite JDBC driver`, `lombok`, and test dependencies.
 - Gradle plugins beyond Spring Boot include `com.netflix.dgs.codegen` and `com.diffplug.spotless`.
- `src/main/resources/application.properties`
 - Data source: `sqlite dev.db` configured via `spring.datasource.url=jdbc:sqlite:dev.db`
 - Jackson setting: `spring.jackson.deserialization.UNWRAP_ROOT_VALUE=true`

- JWT settings: jwt.secret and jwt.sessionTime present (values in file)
- MyBatis configuration properties present (cache-enabled, default-statement-timeout, map-underscore-to-camel-case, use-generated-keys, type-handlers-package, mapper-locations)
- Logging level entries as noted in Logging Implementation
- .github/workflows/gradle.yml
 - CI: GitHub Actions workflow that runs ./gradlew clean test on push and pull_request events

Files and artifacts not found in the repository evidence:

- No pom.xml at project root
- No logback.xml or logback-spring.xml present in evidence
- No Dockerfile or docker-compose files present

Representative Code Evidence

The following excerpts are drawn verbatim or summarized from repository files to demonstrate the logging- and audit-related evidence present in the codebase.

1. application.properties — logging entries and related config:

```
logging.level.io.spring.infrastructure.mybatis.readservice.
ArticleReadService=DEBUG
logging.level.io.spring.infrastructure.mybatis.mapper=DEBUG

spring.datasource.url=jdbc:sqlite:dev.db
spring.datasource.driver-class-name=org.sqlite.JDBC
spring.jackson.deserialization.UNWRAP_ROOT_VALUE=true

jwt.secret=nRvyYC4soFxBdZ-F-5Nzz5USXstR1YlsTd-mA0aKtI9HulriGrtkf-
TiuDapkLiUCog03J0K7kwZisrHp6wA
jwt.sessionTime=86400
```

2. build.gradle — Spring Boot plugin and relevant plugins/dependencies:

```
plugins {
    id 'org.springframework.boot' version '2.6.3'
    id 'io.spring.dependency-management' version '1.0.11.RELEASE'
    id 'java'
    id "com.netflix.dgs.codegen" version "5.0.6"
    id "com.diffplug.spotless" version "6.2.1"
}

sourceCompatibility = '11'

dependencies {
    implementation 'org.springframework.boot:spring-boot-starter-web'
    implementation 'org.springframework.boot:spring-boot-starter-validation'
    implementation 'org.springframework.boot:spring-boot-starter-hateoas'
    implementation 'org.springframework.boot:spring-boot-starter-security'
    implementation 'org.mybatis.spring.boot:mybatis-spring-boot-starter:2.2.2'
    ...
}
```

3. WebSecurityConfig.java — example configuration class with no logger usage:

```

@Configuration
@EnableWebSecurity
public class WebSecurityConfig extends WebSecurityConfigurerAdapter {
    @Bean
    public JwtTokenFilter jwtTokenFilter() {
        return new JwtTokenFilter();
    }

    @Bean
    public PasswordEncoder passwordEncoder() {
        return new BCryptPasswordEncoder();
    }

    @Override
    protected void configure(HttpSecurity http) throws Exception {
        http.csrf().disable()...
        http.addFilterBefore(jwtTokenFilter(), UsernamePasswordAuthenticationFilter.
class);
    }

    @Bean
    public CorsConfigurationSource corsConfigurationSource() { ... }
}

```

Note: The provided configuration classes (WebSecurityConfig, JacksonCustomizations, MyBatisConfig) do not include logger declarations or log statements in the supplied excerpts.

Risks and Observations

All statements below are derived only from repository evidence.

- Limited logging configuration scope:
 - The only explicit logging configuration found is two logging.level entries in application.properties scoped to MyBatis-related packages. There is no broader logging configuration evident (for example, no root logging level or additional package-level entries).
- No explicit logging configuration file found:
 - No logback.xml or logback-spring.xml was present in the evidence. If the application relies on framework defaults, that is not specified in the repository files examined.
- Absence of observable logger usage in provided classes:
 - The provided configuration classes do not demonstrate use of logger instances or log statements. This suggests either logging is implemented elsewhere in code not provided, or logging statements are limited/non-existent in these classes.
- No evidence of an audit subsystem:
 - There is no explicit audit code, audit tables, interceptors, or events visible in the provided repository artifacts.
- Sensitive material in configuration:
 - A JWT secret value is present in application.properties. The repository-level presence of a secret value is observable in the evidence. The repository does not show any redaction or externalization for that secret in the provided files.
- CI runs tests only:
 - The GitHub Actions workflow runs ./gradlew clean test; there is no evidence in the workflow of log collection, artifact upload of logs, or environment-specific logging configuration in CI.

Recommendations

All recommendations below are proposed future improvements and are explicitly framed as recommendations; they are not implemented in the current repository evidence.

- Centralize and document logging configuration:
 - Add an explicit logging configuration file (for example, a Logback configuration such as logback-spring.xml) to codify appenders, encoders (structured JSON if desired), and root and package-level logging policies. This makes logging behavior explicit in the repository.
- Broaden and standardize logging levels:
 - Evaluate and document logging level defaults (including a root level) and extend package-level settings beyond the two MyBatis entries where application observability requires it.
- Introduce explicit logger usage where appropriate:
 - Ensure service, repository, and entry-point classes include measured, structured log statements at appropriate levels. Prefer parameterized logging to minimize string concatenation overhead.
- Implement audit capabilities only where required and explicitly:
 - If audit trails are a requirement, implement a dedicated, explicit audit mechanism (audit events, persistence, and retrieval APIs). Any such implementation should be added as explicitly named code and configuration artifacts in the repository so it can be evidenced and reviewed.
- Externalize and protect secrets:
 - Remove secrets from repository-tracked files and externalize them to environment variables or a secrets management mechanism. Ensure that secrets are not inadvertently logged at runtime.
- Add tests or checks for logging and secrets:
 - Consider adding static checks or unit tests that detect presence of secrets in configuration files and validate logging configuration presence as part of CI.

End of document.